Heap overflows: overview

The heap is the region of memory that a program uses for dynamically allocated data.

- The runtime or operating system provides memory management for the heap.
- With explicit memory management, the programmer uses system calls to allocate and deallocate regions of memory.

Memory allocation in C

malloc(size) tries to allocate a space of size bytes.

- It returns a pointer to the allocated region
- . . . of type void* which the programmer can cast to the desired pointer type
- or it fails and returns a NULL pointer
- The memory is uninitialized so should be written to before being read from

Question. Which points above contribute to (memory) unsafe behaviour in C?

Memory allocation in C

calloc(size) behaves like malloc(size) but it also initialises the memory, clearing it to zeroes.

Question. Suppose we allocate a string buffer and immediately assign the empty string to it. What security reason may there be to prefer calloc() over malloc()?

Memory allocation in C

free(*ptr) frees the previously allocated space at ptr.

- No return value (void)
- If it fails (*ptr a non-allocated value), what happens?
  - if *ptr is NULL, nothing
  - “undefined” otherwise, i.e., program may abort, or might carry on and let bad things happen
- What happens if *ptr is dereferenced after being freed?
  - depends on behaviour of allocator

Question. Suppose we accidently call free(*ptr) before the final dereference of *ptr but before another call to malloc(). Is that safe?

Simple heap variable attack

Without memory safety, heap-allocated variables may overflow from one to another.

```
char *user = (char *)malloc(sizeof(char)*8);
char *adminuser = (char *)malloc(sizeof(char)*8);
strcpy(adminuser, "root");

if (argc > 1)
  strcpy(user, argv[1]);
else
  strcpy(user, "guest");

/* Now we'll do ordinary operations as "user" and create
sensitive system files as "adminuser" */
```

Question. Is it possible to overflow user and change admin user?

Simple heap variable attack

Problem: how do we know where the allocations will be made?

- Heap allocator is free to allocate anywhere, not necessarily in adjacent memory

Let's investigate what happens on Linux x86, glibc. (for a particular version, on a particular day, . . .)

```c
#include <stdlib.h>
#include <string.h>
#include <stdio.h>

void main(int argc, char *argv[]) {
  char *user = (char *)malloc(sizeof(char)*8);
  char *adminuser = (char *)malloc(sizeof(char)*8);

  strcpy(adminuser, "root");

  if (argc > 1)
    strcpy(user, argv[1]);
  else
    strcpy(user, "guest");

  printf("User is at %p, contains: %s\n", user, user);
  printf("Admin user is at %p, contains: %s\n", adminuser, adminuser);
}
```

Simple heap variable attack

```
$ gcc useradminuser.c -o useradminuser.out
$ ./useradminuser.out
User is at 0x950400, contains: guest
Admin user is at 0x950418, contains: root
```

```
$ ./useradminuser.out
User is at 0x948300, contains: guest
Admin user is at 0x948318, contains: root
```

```
$ ./useradminuser.out mihai
User is at 0x865400, contains: mihai
Admin user is at 0x865418, contains: root
```

- Buffers not adjacent, there's some extra space
- Addresses not identical each run
- **But admin user is stored higher in memory!**

Let’s try overflowing….

```
$ ./useradminuser.out super........mihai
User is at 0x940500, contains: super........mihai
Admin user is at 0x940518, contains: ai
```

Let’s try overflowing….

```
$ ./useradminuser.out super........mihai
User is at 0x9405008, contains: super........mihai
Admin user is at 0x9405018, contains: au
```

Count more carefully:

```
$ ./useradminuser.out superDEF01234567mihai
User is at 0x9f0b008, contains: superDEF01234567mihai
Admin user is at 0x9f0b018, contains: mihai
```

Whoa!

Let's try overflowing....

```
$ ./useradminuser.out super........mihai
User is at 0x9405008, contains: super........mihai
Admin user is at 0x9405018, contains: au
```

Count more carefully:

```
$ ./useradminuser.out superDEF01234567mihai
User is at 0x9f0b008, contains: superDEF01234567mihai
Admin user is at 0x9f0b018, contains: mihai
```

Whoa!

Question. Can you think of a way to prevent this attack?